Kirin Autonomous Chess

Hardware Integration Guide

Connecting the Raspberry Pi to the Physical Chess Board

Strydr Silverberg & Colin Dake

Colorado School of Mines
2026 ProtoFund

April 24, 2026

A step-by-step guide to wiring, configuring, and testing the Kirin autonomous chess system on a Raspberry Pi.

Contents

1 Overview **4**
- 1.1 What This Document Covers . . . 4
- 1.2 System Architecture . . . 4
- 1.3 Runtime Move Pipeline . . . 4
- 1.4 Prerequisites . . . 5

2 Building the Software **6**
- 2.1 Clone and Build . . . 6
- 2.2 Install Dependencies . . . 6
- 2.3 Run the Tests . . . 6

3 Wiring the Hall Effect Sensors **8**
- 3.1 Sensor Overview . . . 8
- 3.2 Multiplexer Wiring . . . 8
  - 3.2.1 Shared Select Lines (Pi → All 6 Muxes) . . . 8
  - 3.2.2 Signal Output Lines (Each Mux → Pi) . . . 8
  - 3.2.3 Power and Enable . . . 9
- 3.3 Sensor-to-Channel Mapping . . . 9
  - 3.3.1 Full Board Sensor Map . . . 10
  - 3.3.2 Storage Zone Sensor Map . . . 10
- 3.4 A3144 Sensor Circuit . . . 11

4 Connecting the Gantry **12**
- 4.1 GRBL Connection . . . 12
- 4.2 GRBL Settings . . . 12
- 4.3 Electromagnet Control . . . 13
- 4.4 GRBL Motion Mode Setup . . . 13
- 4.5 Physical Coordinate System . . . 13

5 Operator UI: OLED Display and Buttons **15**
- 5.1 OLED Display . . . 15
- 5.2 Operator Buttons . . . 15

6 First Power-On Procedure **16**
- 6.1 Step 1: Verify GPIO Chip . . . 16
- 6.2 Step 2: Start Kirin in Physical Mode . . . 16
- 6.3 Step 3: Run Sensor Diagnostics . . . 16
- 6.4 Step 4: Quick Scan . . . 17
- 6.5 Step 5: Test the Gantry . . . 17
- 6.6 Step 6: Gantry Coordinate Calibration . . . 17

7 Running Your First Game **19**
- 7.1 Stage 1: Manual Move Entry (No Sensors) . . . 19

7.2 Stage 2: Sensor-Based Play 19
7.2.1 What to Test 19
7.3 Stage 3: Full Game 20

8 Troubleshooting **21**
8.1 Sensor Issues 21
8.2 Gantry Issues 22
8.3 Move Detection Issues 22

9 GPIO Pin Reference **23**

10 Software Command Reference **25**
10.1 Command-Line Validation Modes 25
10.2 Physical Mode Commands 25

11 Tunable Parameters **27**

Chapter 1

Overview

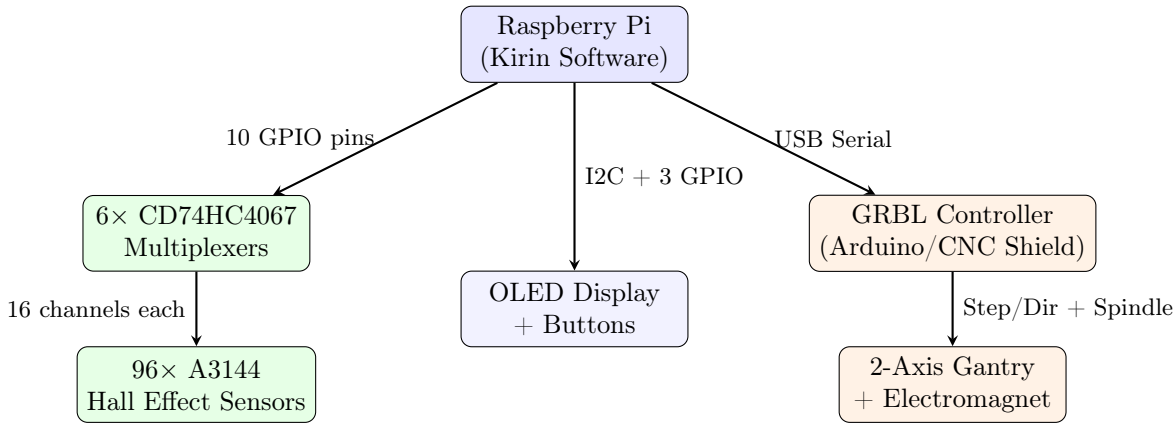
1.1 What This Document Covers

This guide walks through the complete process of connecting the Kirin software to the physical chess board hardware. By the end, you will have a Raspberry Pi running the Kirin engine, reading 96 hall effect sensors through multiplexers, driving an OLED status display, reading operator buttons, and controlling a gantry system via GRBL over serial.

The system has four hardware subsystems that connect to the Pi:

1. **Hall Effect Sensors (96 total)** — detect piece presence on the board and in the two storage zones. Read through 6 multiplexers using 10 GPIO pins.
2. **Operator UI** — a 128×64 I2C OLED for status messages plus Start, Stop, and Reset buttons on dedicated GPIO pins.
3. **GRBL Gantry Controller** — moves pieces on the physical board via an electromagnet on a 2-axis gantry. Communicates over USB serial.
4. **Electromagnet** — controlled by the GRBL board via the spindle output (M3/M5 commands).

1.2 System Architecture



1.3 Runtime Move Pipeline

The physical game loop intentionally keeps a single software-to-hardware path for real hardware, dry-run, and simulation. For the common case where the human starts as white, one full cycle is:

1. **Human moves a piece**: `BoardScanner::waitForLegalMove()` polls the board and storage muxes until the board or storage state differs from the synchronized baseline.

2. **Sensors stabilize**: board and storage are debounced together. A candidate state must remain identical for `DEBOUNCE_SCANS` reads separated by `DEBOUNCE_DELAY_MS`. The scanner never auto-commits after a timeout; unrecognized states only trigger an alert and the loop keeps waiting.

3. **Move is matched to engine format**: `matchLegalMove()` compares the stable occupancy delta against the engine's legal move list. Captures are disambiguated with the exact storage slot that gained a piece and `PieceTracker`'s square-to-slot identity map.

4. **Engine state advances**: the matched encoded move is passed to `makeMove()`, then `GameController::updateTracker()` keeps per-piece slot identity synchronized.

5. **Engine responds**: `searchPosition()` selects `bestMove`. `GameController::executeEngineMove()` converts it to a `PhysicalMove`, builds a `MovePlan`, and sends that plan to the gantry controller.

6. **Blockers move out of the way**: each `RelocationPlan` is executed with magnet on/off dwell time around the move.

7. **Primary piece moves**: the engine-selected piece follows the planned path from source to destination. Captures require an exact storage slot before any capture G-code is emitted.

8. **Blockers restore**: parked blockers are returned in reverse order. When a restoration path exists, the gantry follows it; only an empty restoration path falls back to a direct move.

9. **Board is verified and re-baselined**: after successful hardware execution, the engine state, physical occupancy model, scanner board baseline, and scanner storage baseline are synchronized before waiting for the next human move.

Note: The storage baseline used for a human turn is the last synchronized baseline, not a fresh scan taken after the prompt. This avoids a race where a fast player capture could accidentally become the new baseline instead of being detected as the move.

Warning: Do not bypass `GameController::executeEngineMove()` for real games. It is the boundary that enforces exact capture-slot routing, special-move sequencing, and physical-board state updates. The lower-level G-code helpers are useful for tests and diagnostics only.

1.4 Prerequisites

Item	Details
Raspberry Pi	Pi 4 or Pi 5 (any RAM)
OS	Raspberry Pi OS (64-bit recommended)
Compiler	GCC with C++17 support
CMake	Version 3.10 or later
libgpiod	`sudo apt install libgpiod-dev`
Serial port	USB connection to GRBL controller

Table 1.1: Required components

Chapter 2

Building the Software

2.1 Clone and Build

```
# Clone the repository
git clone https://github.com/strvdr/kirin-autonomous-chess.git
cd kirin-autonomous-chess/kirin

# Create build directory
mkdir build && cd build

# Configure with GPIO support
cmake .. -DCMAKE_BUILD_TYPE=Release \
         -DKIRIN_ENABLE_GPIOD=ON

# Build
cmake --build . --parallel
```

Warning: The `-DKIRIN_ENABLE_GPIOD=ON` option is essential on the Raspberry Pi. Without it, the board scanner and button controller compile in stub mode and GPIO calls are no-ops. The build will succeed either way, but sensors and buttons will not work without GPIO support enabled.

2.2 Install Dependencies

```
# Install libgpiod for GPIO access (no root required at runtime)
sudo apt install libgpiod-dev

# Verify installation
dpkg -l | grep libgpiod
```

2.3 Run the Tests

Before connecting any hardware, verify the software is correct:

```
cd build

# Run all tests
ctest --output-on-failure

# Or run individually
./board_scanner_test      # Sensor move detection logic
./ui_controller_test      # OLED/button controller logic
./game_controller_test    # Engine-to-physical bridge
./board_interpreter_test  # Path planning
```

```
./engine_test              # Engine validation
./captured_piece_test      # Capture encoding
```

All tests should pass. The board scanner tests run entirely in simulation mode and do not require GPIO hardware.

Chapter 3

Wiring the Hall Effect Sensors

3.1 Sensor Overview

The system uses 96 A3144 hall effect sensors to detect the presence of magnets embedded in the chess pieces. The sensors are digital (output LOW when a magnet is detected, HIGH when no magnet is present) and require a 10K pull-up resistor each.

The 96 sensors are organized as follows:

Mux	Coverage	Channels	GPIO (SIG)
0	Black storage zone, left of a-file	0–15	GPIO 5
1	Board files a–b	0–15	GPIO 6
2	Board files c–d	0–15	GPIO 13
3	Board files e–f	0–15	GPIO 19
4	Board files g–h	0–15	GPIO 26
5	White storage zone (16 slots)	0–15	GPIO 16

Table 3.1: Multiplexer assignments

3.2 Multiplexer Wiring

All six CD74HC4067 multiplexers share four address/select lines. The Pi sets these four lines (S0–S3) to select a channel (0–15), then reads the SIG output from each mux simultaneously.

3.2.1 Shared Select Lines (Pi → All 6 Muxes)

Select Line	Pi GPIO (BCM)	Mux Pin
S0	GPIO 17	Pin 10 (S0) on all muxes
S1	GPIO 27	Pin 11 (S1) on all muxes
S2	GPIO 22	Pin 14 (S2) on all muxes
S3	GPIO 23	Pin 15 (S3) on all muxes

Table 3.2: Select line wiring (active on all 6 muxes)

3.2.2 Signal Output Lines (Each Mux → Pi)

Each mux has a single SIG output that goes to its own GPIO pin on the Pi:

Mux	SIG → Pi GPIO	Direction
0	GPIO 5	Input (Pi reads)
1	GPIO 6	Input
2	GPIO 13	Input
3	GPIO 19	Input
4	GPIO 26	Input
5	GPIO 16	Input

Table 3.3: Signal output wiring

3.2.3 Power and Enable

- Connect VCC on each mux to 3.3V (the Pi's GPIO operates at 3.3V).
- Connect GND on each mux to the Pi's GND.
- Connect the EN (enable) pin on each mux to GND (active-low enable, so grounding it keeps the mux always enabled).

Warning: Do not power the muxes from 5V. The CD74HC4067 will work at 3.3V, and connecting a 5V signal to the Pi's GPIO pins can damage the Pi.

3.3 Sensor-to-Channel Mapping

Muxes 1–4 scan the 64 board squares. Each board mux covers two adjacent files. Channels 0–7 map to the first file in that pair from rank 8 down to rank 1; channels 8–15 map to the second file in that pair from rank 8 down to rank 1.

Mux	Files	Channel Pattern
1	a–b	C0=a8, C1=a7, . . . , C7=a1; C8=b8, . . . , C15=b1
2	c–d	C0=c8, C1=c7, . . . , C7=c1; C8=d8, . . . , C15=d1
3	e–f	C0=e8, C1=e7, . . . , C7=e1; C8=f8, . . . , C15=f1
4	g–h	C0=g8, C1=g7, . . . , C7=g1; C8=h8, . . . , C15=h1

Table 3.4: Board mux channel-to-square mapping

Muxes 0 and 5 scan the storage zones. Mux 0 is black storage to the left of the a-file; mux 5 is white storage to the right of the h-file. In both zones, channels 0–7 are the inner column immediately adjacent to the board, and channels 8–15 are the outer pawn column.

Channel	Inner Column Slot	Channel	Outer Column Slot
0	R1	8	P1
1	R2	9	P2
2	B1	10	P3
3	B2	11	P4
4	N1	12	P5
5	N2	13	P6
6	Q	14	P7
7	K	15	P8

Table 3.5: Storage mux channel-to-slot mapping for both mux 0 and mux 5

3.3.1 Full Board Sensor Map

The diagram below shows the complete mapping from every board square to its mux and channel. Each cell shows **M*x*:C*y*** where x is the mux number (1–4) and y is the channel (0–15). The mux select address (S3 S2 S1 S0) is shown for each channel on the right.

	a	b	c	d	e	f	g	h	
8	**M1:C0** a8	**M1:C8** b8	**M2:C0** c8	**M2:C8** d8	**M3:C0** e8	**M3:C8** f8	**M4:C0** g8	**M4:C8** h8	
7	**M1:C1** a7	**M1:C9** b7	**M2:C1** c7	**M2:C9** d7	**M3:C1** e7	**M3:C9** f7	**M4:C1** g7	**M4:C9** h7	**Mux 1 a–b**
6	**M1:C2** a6	**M1:C10** b6	**M2:C2** c6	**M2:C10** d6	**M3:C2** e6	**M3:C10** f6	**M4:C2** g6	**M4:C10** h6	**Mux 2 c–d**
5	**M1:C3** a5	**M1:C11** b5	**M2:C3** c5	**M2:C11** d5	**M3:C3** e5	**M3:C11** f5	**M4:C3** g5	**M4:C11** h5	
4	**M1:C4** a4	**M1:C12** b4	**M2:C4** c4	**M2:C12** d4	**M3:C4** e4	**M3:C12** f4	**M4:C4** g4	**M4:C12** h4	**Mux 3 e–f**
3	**M1:C5** a3	**M1:C13** b3	**M2:C5** c3	**M2:C13** d3	**M3:C5** e3	**M3:C13** f3	**M4:C5** g3	**M4:C13** h3	
2	**M1:C6** a2	**M1:C14** b2	**M2:C6** c2	**M2:C14** d2	**M3:C6** e2	**M3:C14** f2	**M4:C6** g2	**M4:C14** h2	**Mux 4 g–h**
1	**M1:C7** a1	**M1:C15** b1	**M2:C7** c1	**M2:C15** d1	**M3:C7** e1	**M3:C15** f1	**M4:C7** g1	**M4:C15** h1	

Select address (S3 S2 S1 S0) for each channel:

C0: 0000	C1: 0001	C2: 0010	C3: 0011	C4: 0100	C5: 0101	C6: 0110	C7: 0111
C8: 1000	C9: 1001	C10: 1010	C11: 1011	C12: 1100	C13: 1101	C14: 1110	C15: 1111

3.3.2 Storage Zone Sensor Map

The two storage zone muxes each have 16 channels mapped to exact piece storage slots. The black storage zone is left of the a-file on mux 0. The white storage zone is right of the h-file on mux 5. Each zone uses the same top-to-bottom order: R1, R2, B1, B2, N1, N2, Q, K in the inner column and P1–P8 in the outer column.

Black Storage — Mux 0 (GPIO 5)

C0: R1	C8: P1
C1: R2	C9: P2
C2: B1	C10: P3
C3: B2	C11: P4
C4: N1	C12: P5
C5: N2	C13: P6
C6: Q	C14: P7
C7: K	C15: P8

White Storage — Mux 5 (GPIO 16)

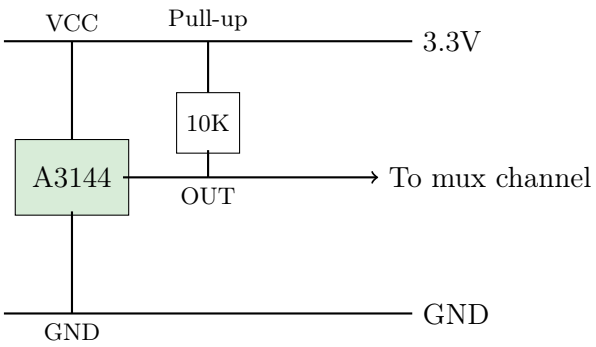
C0: R1	C8: P1
C1: R2	C9: P2
C2: B1	C10: P3
C3: B2	C11: P4
C4: N1	C12: P5
C5: N2	C13: P6
C6: Q	C14: P7
C7: K	C15: P8

Inner column (C0–C7): R1, R2, B1, B2, N1, N2, Q, K.
Outer column (C8–C15): P1, P2, P3, P4, P5, P6, P7, P8.

Tip: If your physical wiring doesn't match this channel order, the software's idea of the board will diverge from the hardware. Either re-wire the sensors to this map or update `toSquareIndex()` and the storage-slot mapping in `board_scanner.cpp`.

3.4 A3144 Sensor Circuit

Each A3144 sensor has three pins: VCC, GND, and OUTPUT. The output is open-collector, so it needs a pull-up resistor:



When a magnet is present, the A3144 pulls its output LOW. When no magnet is present, the pull-up resistor holds the output HIGH. The software inverts this logic: LOW = piece present, HIGH = empty.

Chapter 4

Connecting the Gantry

4.1 GRBL Connection

The gantry controller (typically an Arduino Uno with a CNC shield running GRBL firmware) connects to the Pi via USB. This appears as a serial port, usually `/dev/ttyUSB0` or `/dev/ttyACM0`.

```
# Find the serial port
ls /dev/ttyUSB* /dev/ttyACM*

# Check permissions (your user must be in the 'dialout' group)
groups $USER | grep dialout

# If not in the group:
sudo usermod -aG dialout $USER
# Log out and back in for this to take effect
```

4.2 GRBL Settings

The Kirin software communicates at 115200 baud (GRBL default). Verify the GRBL settings match the physical gantry dimensions. Connect to GRBL with a serial terminal to check:

```
# Connect with minicom (or screen, picocom, etc.)
minicom -D /dev/ttyUSB0 -b 115200

# In the GRBL console, type:
$$
# This prints all settings
```

Key settings to verify:

Setting	Parameter	Notes
$100	X steps/inch	Must match your stepper + lead screw
$101	Y steps/inch	Must match your stepper + lead screw
$110	X max rate (in/min)	Software uses 1000 in/min default
$111	Y max rate (in/min)	Must be $\geq$ 1000
$120	X acceleration (in/s^2)	Start with 10, increase after testing
$121	Y acceleration (in/s^2)	Start with 10, increase after testing
$130	X max travel (in)	Should cover the calibrated command envelope (currently up to X=19.0)
$131	Y max travel (in)	Should cover the calibrated command envelope (currently up to Y=14.92)
$22	Homing cycle	Set to 1 (enabled)

Table 4.1: Critical GRBL settings

4.3 Electromagnet Control

The electromagnet is controlled through GRBL's spindle output:

- `M3` = Magnet ON (spindle CW — repurposed for electromagnet)
- `M5` = Magnet OFF (spindle stop)

The CNC shield's spindle enable pin drives a MOSFET or relay that controls the electromagnet. Verify this works before running any game:

```
# In the GRBL console:
M3      # Magnet should engage
G4 P2.000  # Wait 2 seconds; GRBL P values are seconds
M5      # Magnet should release
```

> **Note:** Kirin's `dwell(milliseconds)` helper accepts milliseconds but emits GRBL seconds. For example, the default pick/release settle time `dwell(100)` is sent as `G4 P0.100`. Sending `G4 P100` would pause for 100 seconds on GRBL, not 100 milliseconds.

4.4 GRBL Motion Mode Setup

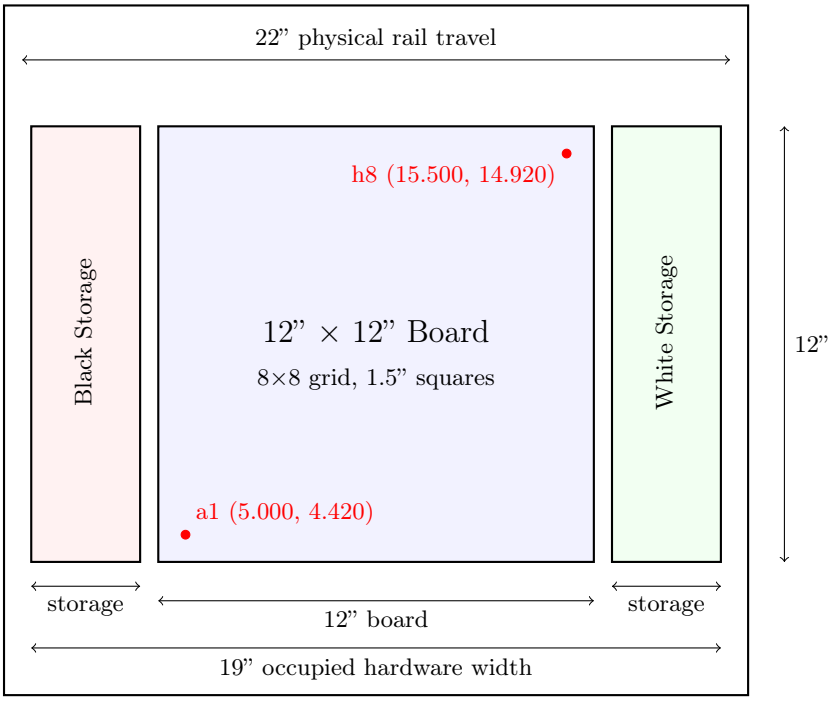
After homing, Kirin sends a deterministic modal setup sequence before normal movement:

```
$H      # Home gantry
G20     # Use inches
G90     # Use absolute coordinates
G94     # Feed rate is units per minute
M5      # Ensure magnet is off
```

> **Warning:** Do not assume the GRBL controller is already in the correct modal state from a previous session. The software's coordinate constants and feed rates are expressed in inches and absolute G54 work coordinates, so `G20`, `G90`, and `G94` must be active before moving pieces.

4.5 Physical Coordinate System

The software maps board squares to physical coordinates in inches. The board layout is:



The physical rails are 22” long. The occupied hardware width is 19”: 3” storage + 0.5” gap + 12” board + 0.5” gap + 3” storage. The coordinate system is the calibrated G54 work frame used by Kirin’s normal `G1` moves. Square a1 is centered at (5.000”, 4.420”) and each square is 1.5” wide. Ranks advance along +X and files advance along +Y, so h8 is centered at (15.500”, 14.920”). The storage rows use the same 1.5” vertical spacing and align with the board file coordinates.

Chapter 5

Operator UI: OLED Display and Buttons

5.1 OLED Display

Kirin supports a 128×64 SSD1306-compatible OLED on the Raspberry Pi's I2C bus. The default software configuration is `/dev/i2c-1` at address `0x3C`.

Signal	BCM GPIO / Header Pin	Connects To
SDA	GPIO 2 / Pin 3	OLED SDA
SCL	GPIO 3 / Pin 5	OLED SCL
3.3V	Pin 1 or Pin 17	OLED VCC
GND	Any Pi ground pin	OLED GND

Table 5.1: OLED I2C wiring

The display controller is implemented in `display_controller.h/.cpp`. Physical mode uses it for high-level status messages: booting, hardware ready, waiting for the human move, engine thinking, executing a move, illegal board state, wrong storage slot, game over, and idle. Use `-oled-test` to initialize the display and show a test screen without starting a physical game.

5.2 Operator Buttons

Kirin supports three active-low buttons using Raspberry Pi internal pull-ups. Pressing a button connects its GPIO line to ground. The button controller is implemented in `button_controller.h/.cpp`.

Button	BCM GPIO / Header Pin	Meaning at Safe Prompt
Start	GPIO 20 / Pin 38	Idle: start `play white`; active game: run `go`
Stop	GPIO 21 / Pin 40	Active game: stop game; idle: exit physical mode
Reset	GPIO 24 / Pin 18	Home the gantry

Table 5.2: Operator button wiring and behavior

Warning: The buttons are high-level UI controls, not an asynchronous emergency stop. They are polled only at safe physical-mode prompts and are not polled during blocking gantry motion, engine search, or `waitForHumanMove`.

Chapter 6

First Power-On Procedure

6.1 Step 1: Verify GPIO Chip

The Pi 5 and Pi 4 use different GPIO chip paths. The software auto-detects, but verify:

```
# Pi 5:
ls /dev/gpiochip4  # Should exist

# Pi 4 and earlier:
ls /dev/gpiochip0  # Should exist

# List all GPIO chips:
gpiodetect
```

6.2 Step 2: Start Kirin in Physical Mode

```
cd build
./kirin --physical /dev/ttyUSB0
```

You should see:

```
Kirin Autonomous Chess System v0.3
Physical Board Mode

[DISPLAY] OLED initialized on /dev/i2c-1 address 0x3C
[BUTTONS] Start=GPIO20 Stop=GPIO21 Reset=GPIO24
Connecting to gantry on /dev/ttyUSB0...
Connected to gantry on /dev/ttyUSB0
Initializing board scanner...
[SCANNER] GPIO initialized successfully (6 muxes: 4 board + 2 storage)
  Select lines: GPIO 17, 27, 22, 23
  Board muxes:  GPIO 6, 13, 19, 26
  Storage muxes: GPIO 5 (black), 16 (white)
Board scanner initialized
Homing gantry...
Homing complete

Ready. Type 'help' for commands.
```

If the scanner fails to initialize, check your wiring and ensure `-DHAS_GPIOD` was used during compilation.

6.3 Step 3: Run Sensor Diagnostics

```
kirin> diag
```

This reads every sensor individually and prints the result. Walk through each square with a magnet (or chess piece) and verify:

- Each channel maps to the correct square
- "PIECE" appears when the magnet is present, "empty" when removed
- No stuck sensors (always reading PIECE or always empty)
- Storage zone sensors respond correctly

6.4 Step 4: Quick Scan

Place all 32 pieces on the board in the starting position, then:

```
kirin> scan
```

You should see:

```
Sensor Scan:
  a b c d e f g h
8 # # # # # # # # 8
7 # # # # # # # # 7
6 . . . . . . . . 6
5 . . . . . . . . 5
4 . . . . . . . . 4
3 . . . . . . . . 3
2 # # # # # # # # 2
1 # # # # # # # # 1
  a b c d e f g h
Pieces detected: 32
```

If any squares are wrong, re-check the wiring for that mux/channel.

6.5 Step 5: Test the Gantry

```
kirin> test
```

This moves the gantry to e4, then d5, engages and disengages the magnet, and returns home. Watch for:

- Smooth motion with no skipped steps
- Correct positioning over the square centers
- Magnet picks up and releases cleanly
- No collisions with pieces or board edges

6.6 Step 6: Gantry Coordinate Calibration

If the gantry doesn't align with the square centers, you need to adjust either the GRBL steps/inch settings or the software constants. Place a piece on a1 and command:

```
kirin> home
```

Then type `test`. The gantry should move to e4 (center of the board). If it's offset, adjust `A1_CENTER_X` and `A1_CENTER_Y` in `gantry_controller.h`, or recalibrate your GRBL $100/$101 steps/inch settings.

Tip: Get a1 and h8 aligned first. Since the coordinate system is linear, if these two corners are correct, all 64 squares will be correct.

Chapter 7

Running Your First Game

7.1 Stage 1: Manual Move Entry (No Sensors)

Start with manual move input to verify the gantry executes moves correctly:

```
kirin> newgame black
```

This sets up the board (gantry moves all pieces from storage to starting squares), then the engine plays black. You type your moves:

```
kirin> move e2e4
kirin> move d2d4
kirin> move g1f3
```

After each of your moves, the engine will think and physically move a piece in response. Watch for correct piece movement, especially during captures (the captured piece should go to the correct storage slot).

7.2 Stage 2: Sensor-Based Play

Once manual mode works, switch to the full sensor experience:

```
kirin> play black
```

Now make your moves by physically moving pieces on the board. The system will detect your move automatically. The terminal will show:

```
Move 1. -- Your turn. Make your move on the board.
Waiting for your move...
Detected move: e2e4
```

7.2.1 What to Test

1. **Normal moves**: Pick up a pawn, place it on a new square.
2. **Pondering**: Pick up a piece, hold it for 10+ seconds, put it back. The system should *not* commit a move.
3. **Take-back**: Pick up one piece, put it back, pick up a different piece and move it. Only the final move should register.
4. **Captures**: Pick up the opponent's piece, place it in the storage zone, then move your piece to the vacated square.
5. **Castling**: Move the king two squares, then move the rook. The system waits until both pieces are in their final positions.
6. **Illegal state**: Move a piece to an illegal square. After 10 seconds, the system will alert you.

7.3 Stage 3: Full Game

Play a complete game to checkmate or stalemate. The system will announce the result:

```
==============================
  CHECKMATE -- White wins!
==============================
```

Chapter 8

Troubleshooting

8.1 Sensor Issues

Symptom	Solution
All sensors read empty	Check mux power (3.3V), verify EN pin is grounded, check select line wiring
One mux reads all empty	Check that mux's SIG output wire to the Pi GPIO pin
Random noise / flickering	Increase `DEBOUNCE_SCANS` (default 3) or `DEBOUNCE_DELAY_MS` (default 50ms) in `board_scanner.h`
Sensors read wrong squares	Your channel-to-square wiring doesn't match the expected layout. Run `diag` and compare against the mapping table. Either re-wire or modify `toSquareIndex()`
Sensors trigger from gantry magnet	The gantry electromagnet is interfering with nearby hall sensors. Add shielding around the gantry magnet, increase debounce, or only scan when the gantry is idle
Scanner init fails	Verify `libgpiod-dev` is installed and the build used `-DHAS_GPIOD`. Check that the GPIO chip exists (`/dev/gpiochip4` on Pi 5, `/dev/gpiochip0` on Pi 4)

Table 8.1: Common sensor issues

8.2 Gantry Issues

Symptom	Solution
"Failed to connect to gantry"	Check USB cable, verify serial port path (`ls /dev/ttyUSB*`), ensure user is in `dialout` group
Homing fails	Verify limit switches are connected and GRBL setting $22=1 (homing enabled)
Gantry moves to wrong position	Recalibrate GRBL steps/inch ($100, $101) or adjust `A1_CENTER_X`/`A1_CENTER_Y` in the software
Magnet doesn't engage	Check GRBL spindle output wiring, test with `M3`/`M5` in a serial console
Pieces dragged/knocked over	Reduce `FEED_RATE` (default 1000 in/min), verify magnet strength is appropriate
GRBL alarm triggered	The software auto-resets on alarm. Check error description in terminal output. Common cause: soft limits exceeded (piece near board edge)

Table 8.2: Common gantry issues

8.3 Move Detection Issues

Symptom	Solution
System doesn't detect my move	Verify the sensor under the destination square works (`diag`). Check that you placed the captured piece in storage (required for capture detection).
System hangs waiting for move	If it's a rare ambiguous capture (queen can capture two different pieces from the same square), fall back to typed input: press Ctrl+C and use `move` command.
"Board state not recognized" alert	You've left the board in a state that doesn't correspond to any legal move for 10+ seconds. Fix piece placement.
Board mismatch after engine move	The gantry dropped a piece. The system will show which squares are wrong. Fix the pieces manually and type `continue`.

Table 8.3: Common move detection issues

Chapter 9

GPIO Pin Reference

Complete pin assignment table using BCM (Broadcom) numbering:

BCM Pin	Direction	Function	Connects To
17	Output	Select S0	S0 on all 6 muxes
27	Output	Select S1	S1 on all 6 muxes
22	Output	Select S2	S2 on all 6 muxes
23	Output	Select S3	S3 on all 6 muxes
5	Input	Mux 0 SIG	Black storage zone
6	Input	Mux 1 SIG	Board files a–b
13	Input	Mux 2 SIG	Board files c–d
19	Input	Mux 3 SIG	Board files e–f
26	Input	Mux 4 SIG	Board files g–h
16	Input	Mux 5 SIG	White storage zone
2	I2C	OLED SDA	OLED display data
3	I2C	OLED SCL	OLED display clock
20	Input	Start button	Active-low, pull-up enabled
21	Input	Stop button	Active-low, pull-up enabled
24	Input	Reset button	Active-low, pull-up enabled

Table 9.1: All GPIO pin assignments for sensors and operator UI

Note: The mux pin assignments can be overridden by passing custom arrays to the `BoardScanner` constructor. The button defaults are defined in `button_controller.h`; the OLED defaults are defined in `display_controller.h`. If you need to change them, update the corresponding default constants and rebuild.

Raspberry Pi Header Pin	**BCM GPIO**
Pin 11	GPIO 17 (S0)
Pin 13	GPIO 27 (S1)
Pin 15	GPIO 22 (S2)
Pin 16	GPIO 23 (S3)
Pin 3	GPIO 2 (OLED SDA)
Pin 5	GPIO 3 (OLED SCL)
Pin 29	GPIO 5 (Mux 0)
Pin 31	GPIO 6 (Mux 1)
Pin 33	GPIO 13 (Mux 2)
Pin 35	GPIO 19 (Mux 3)
Pin 37	GPIO 26 (Mux 4)
Pin 36	GPIO 16 (Mux 5)
Pin 38	GPIO 20 (Start button)
Pin 40	GPIO 21 (Stop button)
Pin 18	GPIO 24 (Reset button)

Table 9.2: BCM to physical header pin mapping

Chapter 10

Software Command Reference

10.1 Command-Line Validation Modes

Use these modes before powering the gantry:

Mode	Purpose
`-simulate`	Interactive move-by-move mode. It uses the same `GameController` to `GrblController` dry-run execution path as physical mode, but prints G-code instead of sending it to serial hardware.
`-dryrun [MOVES [DEPTH]]`	Engine-vs-engine dry run. It exercises setup, path planning, capture-slot routing, blocker relocation, and restoration G-code without connected hardware.

Table 10.1: Command-line validation modes

10.2 Physical Mode Commands

Quick reference for all commands available in physical mode:

Command	Description
`play [white|black]`	Start a sensor-based game. Engine plays the specified color (default: white). Moves are detected automatically via hall effect sensors.
`newgame [white|black]`	Start a game with manual (typed) move entry. Useful as a fallback if sensors aren't working.
`move <uci>`	Manually enter a move in UCI format (e.g., `move e2e4`, `move e7e8q` for promotion).
`go`	Engine makes one move.
`scan`	Scan and display all 96 sensors (board + storage zones).
`diag`	Run full sensor diagnostic — reads each sensor individually with channel-by-channel output.
`board`	Display the engine's internal board position.
`physical`	Display the physical board occupancy tracking.
`fen <fen>`	Set position from a FEN string.
`home`	Home the gantry to (0,0).
`test`	Run gantry hardware test (move to e4, d5, test magnet).
`quit`	Exit the program.

Table 10.2: Physical mode commands

Note: The `-oled-test` command-line flag starts only the OLED test screen. It is useful for verifying display wiring before testing the full physical board. The Start, Stop, and Reset buttons mirror safe prompt actions as documented in the Operator UI chapter.

Chapter 11

Tunable Parameters

These constants in `board_scanner.h` and `gantry_controller.h` may need adjustment based on your specific hardware:

Constant	Default	Purpose
`MUX_SETTLE_US`	10 μs	Time to wait after changing mux address before reading. Increase if readings are noisy.
`DEBOUNCE_SCANS`	3	Number of consecutive identical scans required before accepting a reading as stable.
`DEBOUNCE_DELAY_MS`	50 ms	Delay between debounce scans. Total debounce time = `SCANS` × `DELAY`.
`POLL_INTERVAL_MS`	20 ms	How often to poll sensors during `waitForLegalMove`.
`ILLEGAL_STATE_ALERT_MS`	10,000 ms	How long the board can sit in an unrecognized state before alerting the player.
`FEED_RATE`	1000 in/min	Gantry movement speed. Reduce if the gantry skips steps or knocks pieces over.
`dwell(100)`	100 ms	Pick/release settle time around magnet on/off. Encoded as `G4 P0.100` for GRBL. Increase only if the magnet or pieces need more settle time.
`SQUARE_SIZE`	1.5 in	Physical size of each board square.
`BOARD_SIZE`	12.0 in	Physical size of the 8×8 board area.
`A1_CENTER_X/Y`	5.000 / 4.420 in	Calibrated center of a1 in the G54 work frame.
`CAPTURE_VERTICAL_SPACING`	1.5 in	Storage slot row spacing; matches the board square size.

Table 11.1: Tunable hardware parameters